



DESKRIPSI PERANCANGAN PERANGKAT LUNAK

<Obibi>

untuk:

Dani Hamdani, S.Kom, M.T.

Dipersiapkan oleh:

Daffandri Faiz Zhafransyah	(103012400056)
Gathfan Fadhilah A.S	(103012400255)
Hasnat Ferdiananda	(103012430038)
Rasyid Ridlo	(103012400042)
Faqih Hanif Abdul Rahman	(103012400405)

Program Studi Teknologi Informasi

Fakultas Informatika

Jl. Telekomunikasi 1, Dayeuhkolot Bandung

	Prodi S1- Teknologi Informasi Universitas Telkom	Nomor Dokumen		Halaman
		<i>DPPL-xx</i> <xx:no grp>		<#>/<jml #
		Revisi	<nomor revisi>	Tgl: <isi tanggal>

DAFTAR PERUBAHAN

Revisi	Deskripsi
A	
B	
C	
D	
E	
F	
G	

INDEX TGL	-	A	B	C	D	E	F	G
--------------	---	---	---	---	---	---	---	---

Ditulis oleh								
Diperiksa oleh								
Disetujui oleh								

Daftar Halaman Perubahan

Halaman	Revisi	Halaman	Revisi
---------	--------	---------	--------

--	--	--	--

Daftar Isi

Daftar Isi	2
1. Pendahuluan.....	5
1.1 Tujuan Penulisan Dokumen.....	5
1.2 Lingkup Masalah	5
1.3 Definisi dan Istilah.....	6
1.4 Referensi.....	6
2 Perancangan Global.....	6
2.1 Rencana Lingkungan Implementasi	6
2.2 Deskripsi Arsitektur Perangkat Lunak.....	7
3 Perancangan Rinci	8
3.1 Realisasi Use Case.....	8
3.1.1 Use Case #1 Cek interaksi antarobat	8
3.1.2 Use Case #2 <Input Jadwal Minum Obat>	11
3.1.3 Use Case #3 <Login>	14
3.2 Diagram Kelas Keseluruhan.....	17
3.3 Perancangan Detil Kelas.....	18
3.4 Perancangan Algoritma dan/atau Query	18
4 Matriks Keruntutan (Requirement Traceability Matrix).....	22

Setelah Daftar Isi Boleh ada Daftar Tabel dan Daftar Gambar

1. Pendahuluan

1.1 Tujuan Penulisan Dokumen

Dokumen Deskripsi Perancangan Perangkat Lunak (DPPL) ini disusun untuk memberikan gambaran perancangan teknis dari perangkat lunak Obibi, yang meliputi arsitektur sistem, perancangan use case, desain antarmuka, serta identifikasi kelas dan komponen perangkat lunak yang akan dibangun.

1.2 Lingkup Masalah

Perangkat lunak Obibi merupakan aplikasi pengingat konsumsi obat berbasis kecerdasan buatan yang dirancang untuk membantu lansia dan penyandang disabilitas dalam mengelola jadwal minum obat secara mandiri dan aman. Aplikasi ini menyediakan fitur

pengingat otomatis, pengelolaan data obat, analisis potensi interaksi antarobat, serta asisten virtual berbasis AI.

1.3 Definisi dan Istilah

- A. DPPL : Deskripsi Perancangan Perangkat Lunak, dokumen yang menjelaskan rancangan teknis perangkat lunak
- B. SKPL : Spesifikasi Kebutuhan Perangkat Lunak
- C. Use Case : Skenario interaksi Kebutuhan Perangkat Lunak
- D. UI (User Interface) : Antarmuka yang digunakan pengguna untuk berinteraksi dengan sistem
- E. Boundary Class : Kelas yang menangani interaksi antara sistem dan pengguna
- F. Entity Class : Kelas yang merepresentasikan data dan penyimpanan sistem
- G. Controller Class : Kelas yang mengatur alur logika dan proses bisnis
- H. AI : Artificial Intelligence atau kecerdasan buatan
- I. Agentic AI : Sistem AI yang mampu melakukan pengambilan keputusan dan tindakan secara otonom
- J. GraphRAG : Metode Retrieval-Augmented Generation berbasis graph untuk analisis relasi data

1.4 Referensi

- Dokumen Spesifikasi Kebutuhan Perangkat Lunak (SKPL) Aplikasi Obibi, Program Studi S1 Informatika, Universitas Telkom.
- IEEE Std 830-1993, *IEEE Recommended Practice for Software Requirement Specification*.

2 Perancangan Global

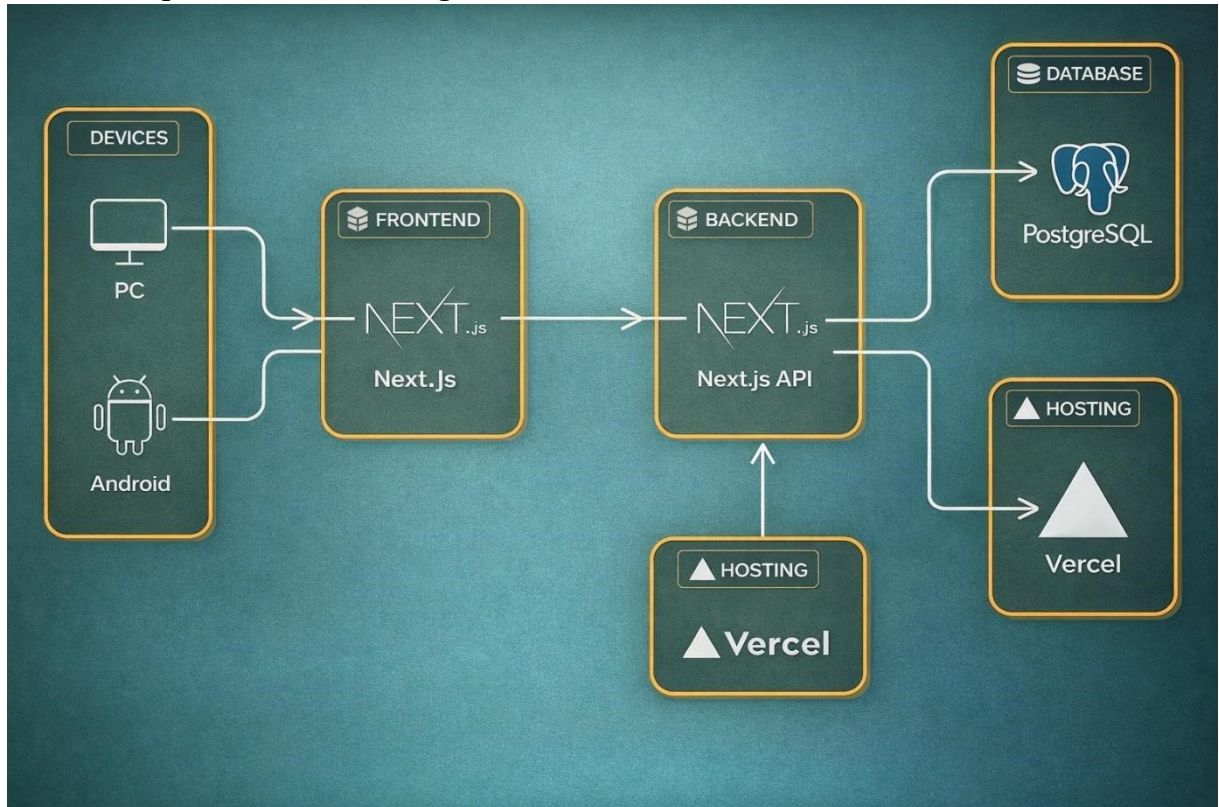
2.1 Rencana Lingkungan Implementasi

Perangkat lunak Obibi dirancang sebagai aplikasi berbasis mobile yang berjalan pada platform Android dan iOS. Sistem ini menggunakan arsitektur client server, di mana aplikasi mobile berperan sebagai client dan backend server bertugas menangani logika bisnis, penyimpanan data, serta integrasi layanan kecerdasan buatan.

Aplikasi mobile berkomunikasi dengan backend melalui protokol HTTPS untuk menjamin keamanan data. Backend sistem dirancang menggunakan pendekatan service based architecture agar mudah dikembangkan dan dipelihara. Sistem memanfaatkan layanan cloud untuk memastikan ketersediaan, skalabilitas, dan reliabilitas aplikasi.

Komponen kecerdasan buatan diintegrasikan pada sisi backend untuk mendukung fitur pengingat cerdas, analisis interaksi antarobat, serta asisten virtual berbasis Agentic AI. Sistem notifikasi pengingat obat dijalankan menggunakan mekanisme penjadwalan otomatis (cron job) dan layanan push notification.

2.2 Deskripsi Arsitektur Perangkat Lunak



Isinya komponen diagram

No	Nama Komponen	Keterangan
1	Mobile Application	Antarmuka pengguna berbasis Android dan iOS untuk mengelola jadwal obat, menerima notifikasi, dan berinteraksi dengan sistem
2	Authentication Service	Modul untuk menangani proses login, logout, dan manajemen autentikasi pengguna
3	User Management Module	Mengelola data profil pengguna dan preferensi personal

4	Medication Management Module	Mengelola data obat, jadwal minum obat, dan siklus pengobatan
5	Reminder Service	Menjalankan penjadwalan pengingat obat dan mengirimkan notifikasi kepada pengguna
6	AI Assistant Module	Menyediakan asisten virtual berbasis AI untuk membantu pengguna mengelola jadwal dan menjawab pertanyaan
7	Drug Interaction Analysis Module	Modul analisis interaksi antarobat menggunakan pendekatan GraphRAG
8	Database System	Menyimpan data pengguna, data obat, jadwal pengobatan, dan histori interaksi
9	Notification Service	Layanan pengiriman notifikasi menggunakan push notification
10	External AI Service	Layanan pihak ketiga untuk pemrosesan kecerdasan buatan dan bahasa alami

3 Perancangan Rinci

3.1 Realisasi Use Case

Berisi TABEL USE CASE sebagai berikut :

No	Nama Use Case	Deskripsi Use Case
#1	Cek interaksi antarobat	Ketika pasien menggunakan use case cek interaksi antarobat, pasien harus sudah menginput obat yang tengah dikonsumsi terlebih dahulu baru bisa mengecek interaksi obat. Sistem akan menggali informasi seputar obat tersebut dari data jurnal klinis dan memberikan kesimpulan terpercaya. Pasien disuruh menghubungi dokter jika data jurnal kurang memadai.
#2	Input jadwal minum obat	Use Case dimulai dari pasien yang menginput jadwal obat ke dalam sistem. Menambahkan detail obat sehingga pasien dapat menerima reminder.
#3	Login	Memungkinkan pengguna mengakses aplikasi Obibi melalui autentikasi akun Google

3.1.1 Use Case #1 Cek interaksi antarobat

3.1.1.1 Use Case Scenario #1 Cek interaksi antarobat

Skenario Use Case #1 :

- i. Pre-Condition:
 - a. Pasien sudah memiliki akun terdaftar

- b. Aplikasi Obibi sudah terpasang pada perangkat pasien ii. Use
- Case Description
- a. Primary Flow
- Pasien membuka menu interaksi
 - Pasien mengecek interaksi antar obat
 - Sistem AI mengecek data jurnal klinis dari DB obat menggunakan metode GraphRAG
 - Sistem mencari konteks sesuai obat pasien dan komplikasinya
 - Sistem membuat kesimpulan dari data
 - Sistem AI mengembalikan kesimpulan dalam bentuk respons
 - Pasien menerima kesimpulan, risiko implikasi, dan suggestion selanjutnya.
- b. Alternative Flow
- Sistem tidak menemukan data obat terkait, menyarankan untuk konsultasi langsung dengan dokter ii. Pasien menerima alert untuk konsultasi langsung dengan dokter
- iii. Post-Condition:
- a. Pasien dapat melihat kesimpulan dan implikasi antar obat yang tengah dikonsumsi

3.1.1.2 UI Design dan Deskripsi Objek UI #1 Cek interaksi antarobat

Bagian ini diisi dengan versi awal prototipe antarmuka untuk per Use Case Selanjutnya, untuk setiap antarmuka/layar, tuliskan spesifikasi detilnya.

Tabel Deskripsi Objek UI

ID. LAYAR	NAMA LAYAR	DESKRIPSI
P005	Halaman Cek interaksi antarobat	Halaman fitur utama di mana pengguna dapat memeriksa potensi interaksi berbahaya antara obat yang baru akan dikonsumsi dengan daftar obat yang saat ini sedang digunakan (Current Medications).

UNTUK MASING – MASING ANTAR MUKA / PAGE dibuatkan spesifikasi detil
Antarmuka XXX: {diisi dengan no. layar atau no gambar rancangan antarmuka}

Page Halaman cek interaksi antarobat

Id_Objek	JENIS	LABEL*	Keterangan**
Sidebar_Menu	Menu Link	Interaksi	Menu navigasi samping. Saat ini dalam keadaan Active (terpilih). Menampilkan halaman fitur interaksi obat.
Tab_Check	Tab button	Cek interaksi	Tombol tab untuk berpindah ke mode pencarian interaksi (sedang aktif). Membedakan tampilan dari tab "Current Medications".
Input_Search	Text Field	Masukkan nama obat	Kolom isian bagi pasien untuk mengetik nama obat baru yang ingin dicek interaksinya (Contoh pada gambar: diisi "Aspirin").

Btn_Check	Button	Check	<i>Tombol aksi. Jika diklik: 1. Sistem memvalidasi input. 2. Sistem mencocokkan obat input dengan Current Medications. 3. Sistem mencari data di jurnal klinis. 4. Menampilkan hasil interaksi.</i>
Lbl_Result	Label	Interaction Results for "[Nama_Obat]"	<i>Judul dinamis yang muncul setelah tombol Check diklik, mengonfirmasi obat apa yang</i>
Alert_Box	Container / Panel	Critical Interaction	<i>Kotak peringatan berwarna merah muda (untuk level Critical). Muncul jika ditemukan interaksi berbahaya. Berisi detail interaksi.</i>
Tag_Meds	Label Tag	Aspirin + Ibuprofen	<i>Menampilkan pasangan obat spesifik yang menyebabkan interaksi.</i>
Txt_Desc	Text Area	Deskripsi	<i>Menjelaskan secara medis mengapa interaksi terjadi (misal: risiko pendarahan gastrointestinal). Data diambil dari jurnal klinis.</i>
Txt_Rec	Text Area	Rekomendasi	<i>Saran tindakan bagi pasien (misal: Hindari kombinasi, hubungi dokter).</i>
Btn_Logout	Button / Link	Log out	<i>Tombol di pojok kiri bawah untuk keluar dari sesi pengguna (akun Hasnat Ferdiananda).</i>

3.1.1.3 Identifikasi Object dan Tipe Kelas #1 Cek interaksi antarobat

Identifikasi object yang terkait dengan use case tersebut.

Kelas pada tahap perancangan berbeda dengan kelas pada tahap analisis.

Gunakan tabel di bawah:

TABEL OBJECT PERANCANGAN

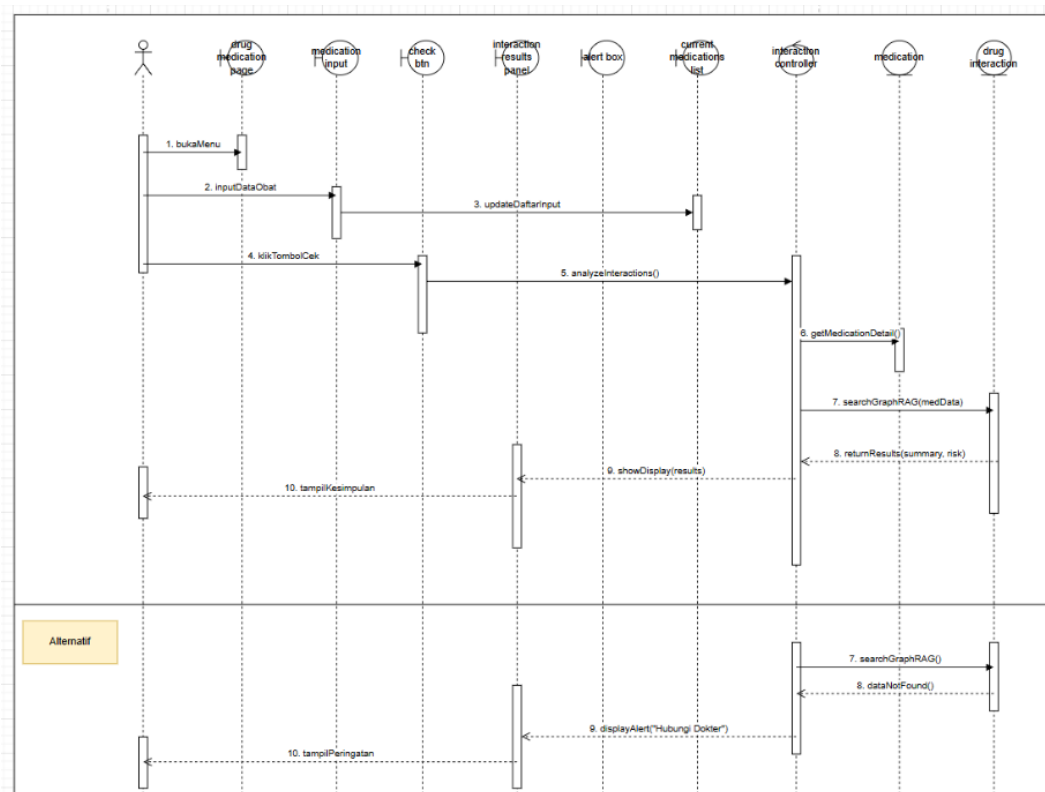
No	Nama Object Baru	Jenis / Tipe Kelas
1	Drug Interaction Page	Boundary
2	Medication Input	Boundary
3	Check Button	Boundary
4	Interaction results panel	Boundary
5	Alert Box	Boundary
6	Current Medications List	Boundary
7	Interaction Controller	Controller

- 8 Medication Entity
- 9 Drug Interaction Entity
- 10 User Entity

*Tipe kelas seperti Boundary(Interface), Entity(Database), Controller

3.1.1.4 Sequence Diagram #1 Cek interaksi antarobat

Buatlah diagram sequence untuk masing - masing use case sesuai skenario.
Skenario harus melibatkan kelas-kelas perancangan yang baru diidentifikasi.



3.1.2 Use Case #2 <Input Jadwal Minum Obat>

3.1.2.1 Use Case Scenario #2 < Input Jadwal Minum Obat >

Skenario Use Case #2 :

- i. Pre-Condition
 - a. Pasien telah login aplikasi Obibi
 - b. Pasien memiliki akun terdaftar
 - c. Aplikasi Obibi sudah terpasang pada perangkat pasien
- ii. Use Case Description
 - b. Primary Flow
 - Pasien membuka halaman Kelola Obat.
 - Pasien menekan tombol Tambah Jadwal pada obat yang dipilih.
 - Sistem menampilkan modal “Tambah Jadwal Minum Obat”. ■ Pasien memilih Tipe Jadwal (default: Setiap Hari).
 - Pasien mengisi Waktu Minum menggunakan format 24 jam.

- (Opsional) Pasien menekan tombol + Tambah Waktu untuk menambahkan waktu minum lain.
 - Pasien memilih Mulai Tanggal pengingat.
 - (Opsional) Pasien mengisi Sampai Tanggal sebagai batas akhir jadwal.
 - Pasien menekan tombol Simpan Jadwal.
 - Sistem memvalidasi seluruh input (format waktu, tanggal, dan kelengkapan data). ■
Sistem menyimpan data jadwal minum obat ke database.
 - Sistem membentuk dan mengaktifkan scheduler / cronjob reminder sesuai jadwal.
 - Sistem menutup modal dan menampilkan jadwal yang telah ditambahkan pada halaman Kelola Obat.
- c. Alternative Flow
- Jika data tidak valid atau tidak lengkap
 - Sistem menampilkan pesan kesalahan
 - Data tidak disimpan ke database
 - Jika penyimpanan data gagal
 - Sistem menampilkan notifikasi gagal menyimpan data
 - Cronjob tidak dibuat
- iv. Post-Condition
- a. Jadwal tersimpan di database
 - b. Cronjob reminder aktif
 - c. Jadwal tampil di halaman

3.1.2.2 UI Design dan Deskripsi Objek UI #2 < Input Jadwal Minum Obat >

Bagian ini diisi dengan versi awal prototipe antarmuka untuk per Use Case Selanjutnya, untuk setiap antarmuka/layar, tuliskan spesifikasi detilnya.

Tabel Deskripsi Objek UI

ID. LAYAR	NAMA LAYAR	DESKRIPSI
UI-02	Tambah Jadwal Minum Obat	Modal dialog untuk mengatur jadwal rutin minum obat menggunakan format waktu 24 jam, termasuk tipe jadwal dan rentang tanggal

UNTUK MASING – MASING ANTAR MUKA / PAGE dibuatkan spesifikasi detil Antarmuka XXX: {diisi dengan no. layar atau no gambar rancangan antarmuka}

UI-02 – Tambah Jadwal Minum Obat

ID Objek	Jenis	Label	Keterangan
TXT01	Label	Tambah Jadwal Minum Obat	Judul modal input jadwal
TXT02	Label	Atur jadwal rutin...	Instruksi penggunaan format waktu 24 jam
DROP01	Dropdown	Tipe Jadwal	Pilihan tipe jadwal (default: Setiap Hari)
TIME01	Time Input	Waktu Minum	Input waktu konsumsi obat (format 24 jam, contoh 08:00)
BTN01	Button	+ Tambah Waktu	Menambahkan field waktu minum tambahan
DATE01	Date Picker	Mulai Tanggal	Menentukan tanggal mulai pengingat
DATE02	Date Picker	Sampai Tanggal (opsional)	Menentukan tanggal akhir pengingat (opsional)

BTN02	Button	Batal	<i>Menutup modal tanpa menyimpan data</i>
BTN03	Button	Simpan Jadwal	<i>Menyimpan jadwal dan memicu proses validasi & penyimpanan</i>

3.1.2.3 Identifikasi Object Baru & Tipe Kelas #2 <Input Jadwal Minum Obat >

Identifikasi object yang terkait dengan use case tersebut.

Kelas pada tahap perancangan berbeda dengan kelas pada tahap analisis.

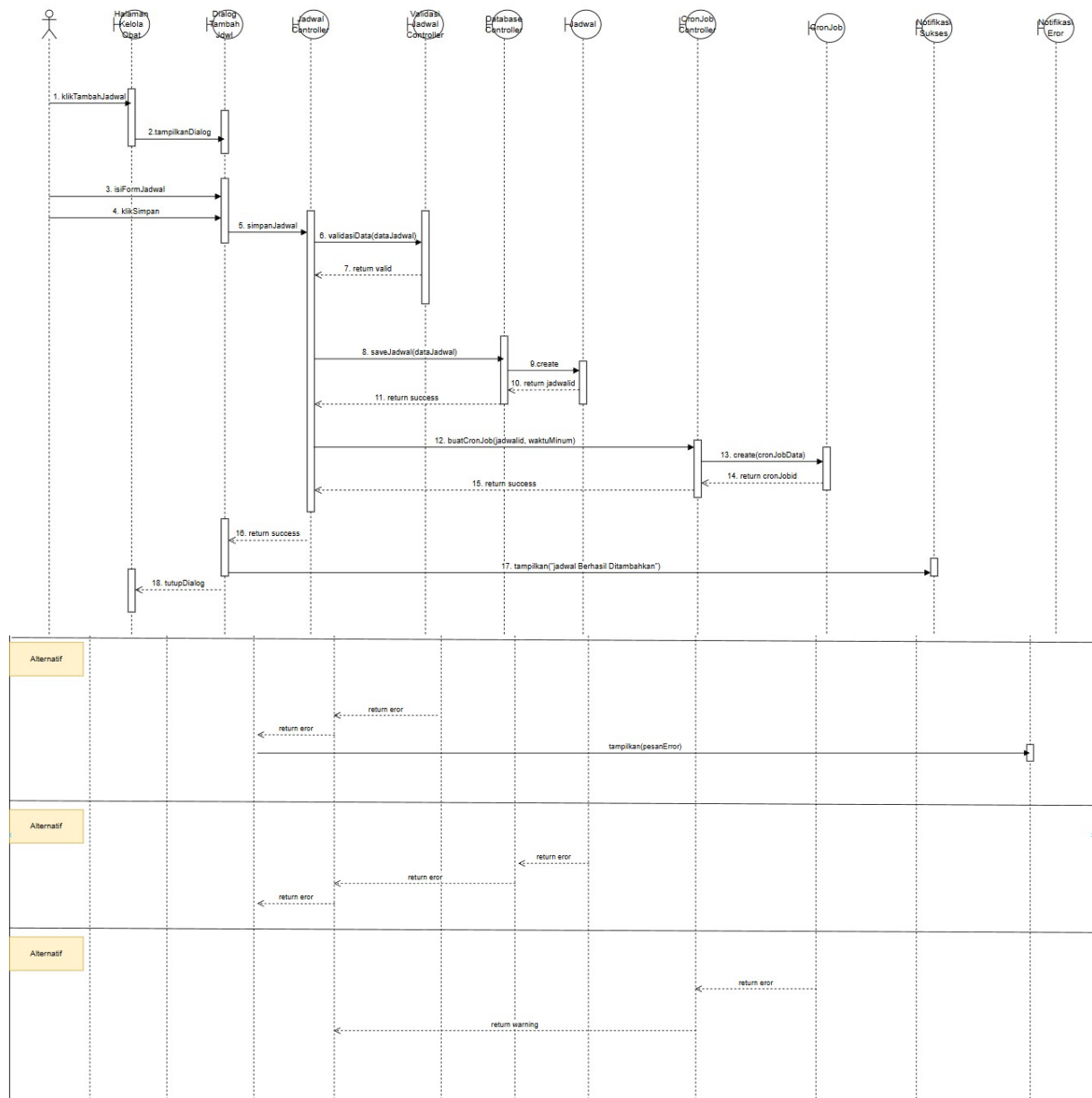
Gunakan tabel di bawah:

TABEL OBJECT PERANCANGAN

No	Nama Object	Jenis / Tipe Kelas
1	JadwalObatUI	Boundary
2	JadwalObatController	Controller
3	JadwalObat	Entity
4	ReminderService	Controller
5	DatabaseService	Entity

**Tipe kelas seperti Boundary(Interface), Entity(Database), Controller*

3.1.2.4 Sequence Diagram #2 < Input Jadwal Minum Obat >



3.1.3 Use Case #3 <Login>

3.1.3.1 Use Case Scenario #3<login>

Skenario Use Case #3 :

- v. Pre-Condition:
 - o Pengguna telah menginstal aplikasi Obibi.
 - o Pengguna memiliki akun Google yang aktif.
 - o Koneksi internet tersedia.
- vi. Use Case Description
 - a. Primary Flow
 - Pasien klik "Tambah Jadwal"
 - Sistem buka modal form

- Pasien isi: Tipe Jadwal, Waktu (HH:MM), Tanggal Mulai, Tanggal Sampai (opsional)
 - Pasien klik "Simpan Jadwal"
 - Sistem validasi input
 - Sistem simpan ke database
 - Sistem buat cronjob reminder
 - Sistem tampilkan notifikasi sukses dan refresh halaman
- b. Alternative Flow
- **Alt 1: Validasi Gagal**
 - Sistem deteksi error (waktu/tanggal invalid)
 - Tampilkan pesan error di field terkait
 - User perbaiki input
 - **Alt 2: Database Error**
 - Gagal simpan ke database
 - Tampilkan toast error "Gagal menyimpan, coba lagi"
 - User dapat retry atau batal
 - **Alt 3: Cronjob Gagal**
 - Jadwal tersimpan tapi cronjob gagal dibuat
 - Sistem log error
 - User tetap dapat notifikasi sukses
 - Background service retry otomatis
- vii. Post-Condition
- Pengguna berhasil login ke dalam sistem.
 - Session pengguna tersimpan.
 - Pengguna dapat mengakses fitur-fitur aplikasi Obibi.

3.1.3.2 UI Design dan Deskripsi Objek UI #3<Login>

Tabel Deskripsi Objek UI

ID. LAYAR	NAMA LAYAR	DESKRIPSI
UI-01	Login Page	Halaman autentikasi pengguna menggunakan akun Google

UNTUK MASING – MASING ANTAR MUKA / PAGE dibuatkan spesifikasi detil *Antarmuka*
 XXX: {diisi dengan no. layar atau no gambar rancangan antarmuka}

Id_Objek	JENIS	LABEL*	Keterangan**
BTN-01	Button	Lanjutkan dengan Google	Mengarahkan pengguna ke proses autentikasi Google
TXT-01	Text	Selamat Datang di Obibi	Menampilkan judul halaman login
MSG-01	Message Box	Error Login	Menampilkan pesan kesalahan apabila autentikasi gagal

3.1.3.3 Identifikasi Object Baru & Tipe Kelas #3 <Login>

Identifikasi object yang terkait dengan use case tersebut.

Kelas pada tahap perancangan berbeda dengan kelas pada tahap analisis.

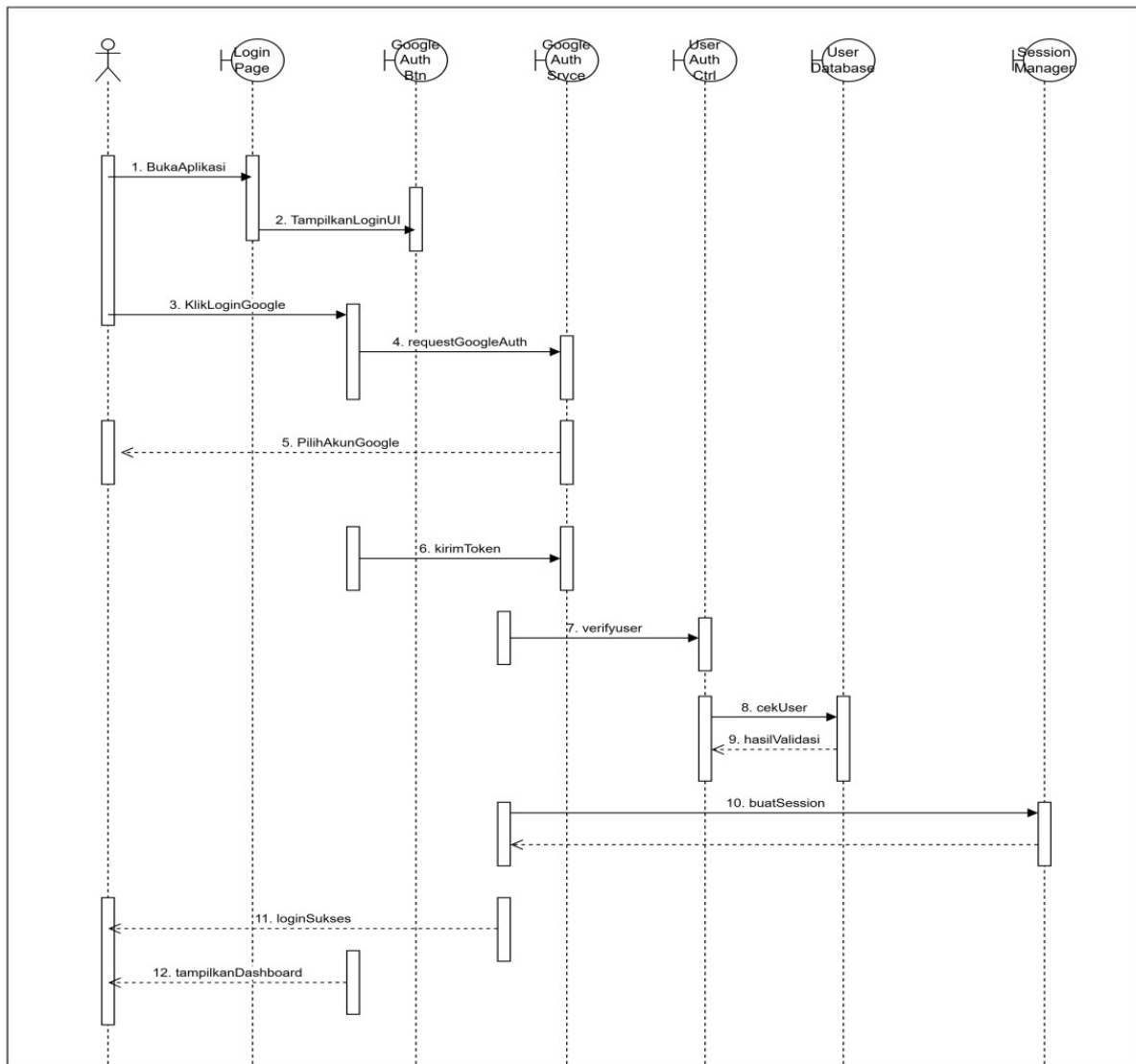
Gunakan tabel di bawah:

TABEL OBJECT PERANCANGAN

No	Nama Object Baru	Jenis / Tipe Kelas
1	Login Page	Boundary
2	GoogleAuthButton	Bondary
3	GoogleAuthService	Controller
4	UserAuthController	Controller
5	User Database	Entity
6	SessionManager	Entity

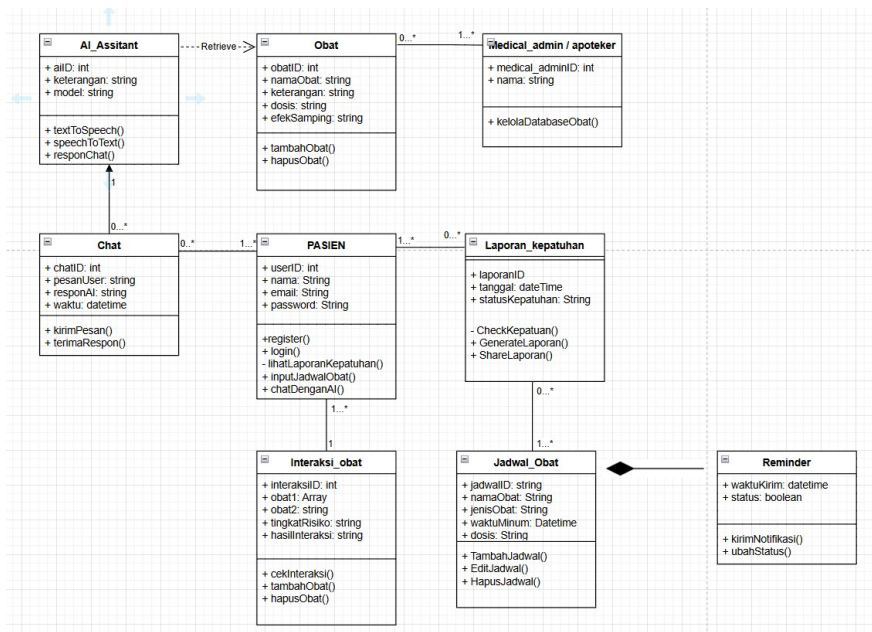
**Tipe kelas seperti Boundary(Interface), Entity(Database), Controller*

3.1.3.4 Sequence Diagram #3 <Login>



3.2 Diagram Kelas Keseluruhan

Bagian ini diisi dengan diagram kelas keseluruhan yang akan digunakan dalam PL menggunakan model MVC



3.3 Perancangan Detil Kelas

Bagian ini diisi dengan daftar keseluruhan kelas yang akan digunakan dalam PL. menggunakan model MVC **TABEL KELAS** :

ID Kelas	Nama Kelas Perancangan	Atribut (visibility)	Method / Operation
	Cek interaksi antarobat	- interaksiID: int (private) - obat1: Array (private) - obat2: string (private) - tingkatRisiko: string (private) - hasilInteraksi: string (private)	+ cekInteraksi(): void + ekstrakRekomendasi: void + mergeInteraksi(): void + generateWarning(): string + generateRekomendasi(): string
	Input Jadwal Minum Obat	- jadwalID: string (private) - namaObat: String (private) - jenisObat: String (private) - waktuMinum: Datetime (private) - dosis: String (private)	+ TambahJadwal(): void + EditJadwal(): void + HapusJadwal(): void + getJadwal(): object

Untuk setiap kelas:

- identifikasi operasi (mengacu pada tanggung-jawab kelas), identifikasi atribut, termasuk visibility-nya

3.4 Perancangan Algoritma dan/atau Query

Bagian ini hanya diisi untuk kerangka algoritma untuk proses-proses yang dianggap cukup penting. Implementasi skeleton code juga sudah dapat dilakukan untuk kelas-kelas yang terdefinisi pada bahasa pemrograman tertentu. Boleh dibuat subbab per kelas.

Algoritma #1

Nama Kelas : cek interaksi antarobat
 Algoritma :

--

```
export class InteractionService {
  static async checkInteractions(
    drugName: string,
    userId?: string
  ): Promise<InteractionCheckResult> {
    const supabase = await createClient();

    // Get user's current medications
    let userMedications: MedicationBasic[] = [];
    if (userId) {
      const { data } = await supabase
        .from("medications")
        .select("id, name, dosage, dosage_unit")
        .eq("user_id", userId);
      userMedications = data || [];
    } else {
      // If no userId, try to get from auth
      const {
        data: { user },
      } = await supabase.auth.getUser();
      if (user) {
        const { data } = await supabase
          .from("medications")
          .select("id, name, dosage, dosage_unit")
          .eq("user_id", user.id);
        userMedications = data || [];
      }
    }

    if (userMedications.length === 0) {
      return {
        drugName,
        userMedications: [],
        interactions: [],
        stats: { total: 0, critical: 0, moderate: 0, mild: 0 },
        safeToUse: true,
        warnings: [
          "No current medications found. Unable to check interactions.",
        ],
        recommendations: [
          "Add your current medications to enable interaction checking.",
        ],
      };
    }

    // Step 1: Quick check using local database
    const medicationNames = userMedications.map((m) => m.name);
    const localInteractions = findInteractions(medicationNames, drugName);

    // Step 2: Enhanced check using RAG for detailed information
    const ragInteractions = await this.checkInteractionsWithRAG(
      drugName,
      medicationNames
    );

    // Step 3: Merge and deduplicate results
    const allInteractions = this.mergeInteractions(
      localInteractions,
      ragInteractions,
      userMedications
    );

    // Step 4: Calculate stats and safety assessment
    const stats = this.calculateStats(allInteractions);
    const safeToUse = stats.critical === 0;
    const warnings = this.generateWarnings(allInteractions, stats);
    const recommendations = this.generateRecommendations(
      allInteractions,
      stats
    );

    return {
      drugName,
      userMedications: userMedications.map((m) => ({
        id: m.id,
        name: m.name,
        dosage: `${m.dosage || ""} ${m.dosage_unit || ""}`.trim(),
      })),
      interactions: allInteractions,
      stats,
      safeToUse,
      warnings,
      recommendations,
    };
  }
}
```

Algoritma #2

Nama Kelas : input jadwal obat

Algoritma :

```
export class ScheduleService {
  static async getSchedulesByMedication(medicationId: string): Promise<MedicationSchedule[]> {
    const supabase = await createClient();

    const { data, error } = await supabase
      .from('medication_schedules')
      .select('*')
      .eq('medication_id', medicationId)
      .order('created_at', { ascending: false });

    if (error) throw error;
    return data || [];
  }

  /**
   * Create new schedule
   */
  static async createSchedule(input: CreateScheduleInput): Promise<MedicationSchedule> {
    const supabase = await createClient();

    const { data, error } = await supabase
      .from('medication_schedules')
      .insert(input)
      .select()
      .single();

    if (error) throw error;

    // Generate reminders for today and upcoming days
    await this.generateRemindersForSchedule(data.id);

    return data;
  }

  /**
   * Update schedule
   */
  static async updateSchedule(
    id: string,
    input: Partial<CreateScheduleInput>
  ): Promise<MedicationSchedule> {
    const supabase = await createClient();

    const { data, error } = await supabase
      .from('medication_schedules')
      .update(input)
      .eq('id', id)
      .select()
      .single();

    if (error) throw error;

    // Regenerate reminders if schedule changed
    await this.generateRemindersForSchedule(id);

    return data;
  }
}
```

Query #1

{Jika mengacu query tertentu, lengkapi query di bawah}

Query :

let { data: medications, error } = await supabase.from('medications').select('name')

Deskripsi: mengambil nama obat dari table medications.

Query #2

{Jika mengacu query tertentu, lengkapi query di bawah}

Query : `const { data, error } = await supabase.from('medication_schedules').insert([ { some_column: 'someValue', other_column: 'otherValue' }, ]) .select()`
Deskripsi: menambah jadwal minum obat ke table medication_schedules.

4 Matriks Keruntutan (Requirement Traceability Matrix)

Mapping requirement dengan Use Case yang direalisasikan

Kode FR	Nama Functional Requirement	Nama Use Case
FR-01	Manajemen Akun Pengguna	Login
FR-02	Pengingat Konsumsi Obat	Terima reminder minum obat
FR-03	Keamanan Data Pengguna	Validasi Data
FR-04	Akurasi Informasi Medis	Kelola DB Informasi Obat
FR-05	Analisis Interaksi Obat	Cek interaksi antarobat
FR-06	Interaksi Berbasis Suara	Text-to-Speech & Speech to text chat
FR-07	Personalisasi Jadwal Pengobatan	Input jadwal minum obat
FR-08	Asisten Virtual Cerdas	Chat dengan Dokter AI-Assistant
FR-09	Mode Tampilan Ramah Disleksia	Setting Mode Dyslexia-friendly
FR-10	Mengelola Data Obat Pengguna	Input obat yang sedang dirasakan